Final Project Report: Multi-threaded TCP Proxy Server

Ada Şevval Sari
Student ID: 220303023
Date: 23 January 2026

Abstract

This report presents the final implementation of a multi-threaded TCP-based proxy server developed in C using the Windows Sockets API (Winsock2). The system successfully handles concurrent client connections by spawning a dedicated thread for each session. It parses incoming HTTP traffic to determine the target destination and supports the `CONNECT` method for tunneling HTTPS traffic transparently. Key features implemented include a robust timeout mechanism using `select()`, rewriting absolute-form HTTP requests, and a metrics system that logs Time-To-First-Byte (TTFB), total transferred bytes, and session duration to the console. The final application operates on port 8080 and demonstrates efficient resource handling and full-duplex data relaying.

Arel Üniversitesi, Müh. Mim. Fak. Bilgisayar Mühendisliği. Türkoba Mah., Erguvan Sok., No:26, K 34537, Tepekent – Büyükçekmece, İstanbul

1 Introduction

The objective of this project was to design and implement a functional proxy server capable of intercepting, parsing, and forwarding network traffic. The completed application fulfills these requirements by acting as an intermediary between a client (e.g., a web browser) and a destination server.

The implementation distinguishes between standard HTTP requests, which require parsing the "Host" header, and HTTPS requests, which utilize the `CONNECT` method to establish an encrypted tunnel. The system is built on a multi-threaded architecture using the Windows API (`CreateThread`), ensuring that the main listener loop remains unblocked while multiple clients are served simultaneously.

Furthermore, the project includes detailed session metrics. For every connection, the server calculates the latency (TTFB) and tracks the volume of data exchanged between the client and the server. This report details the theoretical background, the system design based on the actual code structure, and the performance characteristics of the final software.

2 Theory and Implementation

Winsock2 and Socket Management

The project relies on the `Winsock2` library for network communication. The server initializes the WSA subsystem, binds to a local IP address (127.0.0.1) and port (8080), and enters a listening state. Robust error handling is implemented using `WSAGetLastError()` to diagnose issues during socket creation or binding.

Multi-threading with Windows API

Unlike single-threaded or select-based concurrency models, this implementation uses a thread-per-client model. Upon accepting a connection, the main thread allocates a `ClientContext` structure and immediately spawns a new worker thread via `CreateThread`. This ensures that long-running downloads or stalled connections do not affect the server's ability to accept new clients.

HTTPS Tunneling (CONNECT Method)

For HTTPS, the proxy implements tunneling. When a `CONNECT` request is detected, the server establishes a connection to the target host and replies with "200 Connection Established". From that point onward, the socket acts as a blind relay, forwarding encrypted binary data bi-directionally without inspection.

I/O Multiplexing and Timeouts

To prevent "zombie" connections, the implementation uses the `select()` system call with a defined timeout (configured to 30 seconds). This allows the server to monitor both client and upstream sockets for readability simultaneously. If no data is received within the timeout period, the session is gracefully terminated to free resources.

Metrics Calculation

Performance monitoring is achieved using high-resolution timers (`GetTickCount64`). The application records the start time of the upstream connection and captures the timestamp of the first byte received from the server to calculate the Time-To-First-Byte (TTFB).

3 System Design

Architectural Overview

The system is designed using a procedural approach centered around a main event loop and worker threads. The core components of the implementation are:

- **Main Listener:** Initializes sockets and accepts incoming TCP connections.
- **Client Context:** A dynamic structure that passes socket descriptors to worker threads.
- **Proxy Thread:** The core logic function that determines protocol type (HTTP vs. HTTPS) and manages the relay lifecycle.
- **Metrics Engine:** Embedded within the relay function to log statistics upon session closure.

UML Diagram

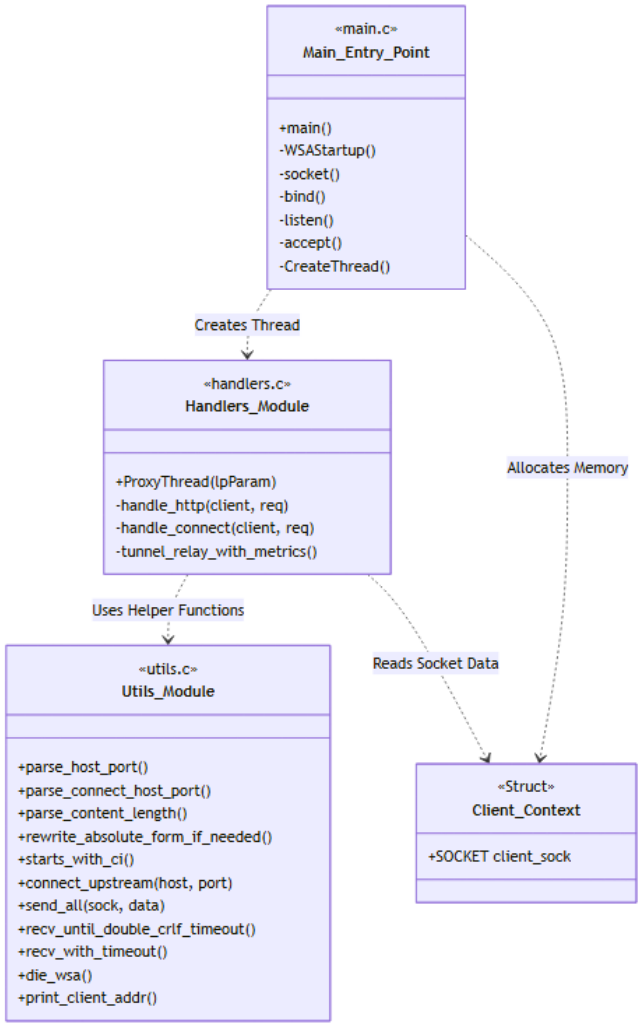

Figure 1: System Architecture Diagram

Operational Logic

- **Connection Phase:** The client connects, and the server parses the initial request headers to extract the hostname and port.
- **Relay Phase:**
  - For **HTTP**, headers are rewritten if necessary, and data is forwarded.
  - For **HTTPS**, a tunnel is established, and the server enters a loop utilizing `select()` to relay raw bytes between sockets.

5 Conclusion and Remarks

The final implementation successfully meets the project requirements. The proxy acts as a transparent intermediary, correctly handling complex web traffic including SSL/TLS streams. The use of multi-threading ensures responsiveness, while the timeout logic prevents resource exhaustion.

Key Achievements:

- Full support for standard HTTP methods and HTTPS CONNECT tunneling.
- Implementation of a custom `recv_until_double_crlf` function to robustly parse HTTP headers.
- Real-time console logging of connection metrics (Target Host, Bytes Sent/Received, Duration).
- Error resilience using WSA error codes and graceful socket closure.

Future iterations could expand on this foundation by adding a configuration file for port settings, implementing access control lists (ACLs) based on IP addresses, or transitioning to an I/O completion port (IOCP) model for higher scalability on Windows.

References

1. Microsoft Docs, *Windows Sockets 2 (Winsock) Application Programming Interface.*
2. Beej Jorgensen, *Beej's Guide to Network Programming.*
3. RFC 7231, *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content.*
4. RFC 2817, *Upgrading to TLS Within HTTP/1.1* (Section on CONNECT).